

ExpressionSet and SummarizedExperiment

```
library(Biobase)
library(GEOquery)
```

```
# If network access is present, the typical way to do this would be this:
geoq <- getGEO("GSE9514")
```

```
## ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE9nnn/GSE9514/matrix/
## Found 1 file(s)
## GSE9514_series_matrix.txt.gz
```

```
## Warning in download.file(myurl, destfile, mode = mode, quiet = TRUE, method
## = getOption("download.file.method.GEOquery")): downloaded length 9477465 !=
## reported length 200
```

```
## File stored at:
## C:\Users\Prasanth\AppData\Local\Temp\RtmpWm1N1C/GPL90.soft
```

```
# if no internet, you can assign to a system file like this
# geoq <- getGEO(filename=system.file("GSE9514_series_matrix.txt.gz", package = "GEOquery"))
names(geoq)
```

```
## [1] "GSE9514_series_matrix.txt.gz"
```

```
e <- geoq[[1]]
```

ExpressionSet

```
dim(e)
```

```
## Features  Samples
##      9335      8
```

```
exprs(e)[1:3,1:3]
```

```
##           GSM241146  GSM241147  GSM241148
## 10000_at    15.33081    9.459372    7.984865
## 10001_at    283.47190  300.729460  270.016080
## 10002_i_at 2569.45360 2382.814700 2711.814500
```

```
dim(exprs(e))
```

```
## [1] 9335    8
```

```
pData(e)[1:3,1:6]
```

```
##                                     title
## GSM241146 hem1 strain grown in YPD with 250 uM ALA (08-15-06_Philpott_YG_S98_1)
## GSM241147  WT strain grown in YPD under Hypoxia (08-15-06_Philpott_YG_S98_10)
## GSM241148  WT strain grown in YPD under Hypoxia (08-15-06_Philpott_YG_S98_11)
##           geo_accession      status submission_date
## GSM241146    GSM241146 Public on Nov 06 2007      Nov 02 2007
## GSM241147    GSM241147 Public on Nov 06 2007      Nov 02 2007
## GSM241148    GSM241148 Public on Nov 06 2007      Nov 02 2007
##           last_update_date type
## GSM241146    Aug 14 2011  RNA
## GSM241147    Aug 14 2011  RNA
## GSM241148    Aug 14 2011  RNA
```

```
dim(pData(e))
```

```
## [1]  8 31
```

```
names(pData(e))
```

```
## [1] "title"           "geo_accession"
## [3] "status"          "submission_date"
## [5] "last_update_date" "type"
## [7] "channel_count"    "source_name_ch1"
## [9] "organism_ch1"     "characteristics_ch1"
## [11] "molecule_ch1"    "extract_protocol_ch1"
## [13] "label_ch1"        "label_protocol_ch1"
## [15] "taxid_ch1"        "hyb_protocol"
## [17] "scan_protocol"    "description"
## [19] "data_processing"  "platform_id"
## [21] "contact_name"     "contact_email"
## [23] "contact_department" "contact_institute"
## [25] "contact_address"  "contact_city"
## [27] "contact_state"    "contact_zip/postal_code"
## [29] "contact_country"  "supplementary_file"
## [31] "data_row_count"
```

```
pData(e)$characteristics_ch1
```

```
##                V2                V3                V4
## BY4742 hem1 delta strain          BY4742          BY4742
##                V5                V6                V7
## BY4742 hem1 delta strain BY4742 hem1 delta strain BY4742 hem1 delta strain
##                V8                V9
##          BY4742 strain          BY4742 strain
## Levels: BY4742 BY4742 hem1 delta strain BY4742 strain
```

```
fData(e)[1:3,1:3]
```

```
##          ID      ORF SPOT_ID
## 10000_at  10000_at YLR331C  <NA>
## 10001_at  10001_at YLR332W  <NA>
## 10002_i_at 10002_i_at YLR333C  <NA>
```

```
dim(fData(e))
```

```
## [1] 9335  17
```

```
names(fData(e))
```

```
## [1] "ID"                "ORF"
## [3] "SPOT_ID"           "Species Scientific Name"
## [5] "Annotation Date"   "Sequence Type"
## [7] "Sequence Source"   "Target Description"
## [9] "Representative Public ID" "Gene Title"
## [11] "Gene Symbol"       "ENTREZ_GENE_ID"
## [13] "RefSeq Transcript ID" "SGD accession number"
## [15] "Gene Ontology Biological Process" "Gene Ontology Cellular Component"
## [17] "Gene Ontology Molecular Function"
```

```
head(fData(e)$"Gene Symbol")
```

```
## [1] JIP3  MID2          RPS25B          NUP2
## 4869 Levels: AC01 ARV1 ATP14 BOP2 CDA1 CDA2 CDC25 CDC3 CDD1 CTS1 ... I14
```

```
head(rownames(e))
```

```
## [1] "10000_at"  "10001_at"  "10002_i_at" "10003_f_at" "10004_at"
## [6] "10005_at"
```

```
experimentData(e)
```

```
## Experiment data
##   Experimenter name:
##   Laboratory:
##   Contact information:
##   Title:
##   URL:
##   PMIDs:
##   No abstract available.
```

```
annotation(e)
```

```
## [1] "GPL90"
```

Summarized Experiment

Install “parathyroidSE” datafile using `BiocInstaller::biocLite("parathyroidSE")`

```
library(parathyroidSE)
```

```
data(parathyroidGenesSE)
se <- parathyroidGenesSE
se
```

```
## class: SummarizedExperiment
## dim: 63193 27
## exptData(1): MIAME
## assays(1): counts
## rownames(63193): ENSG000000000003 ENSG000000000005 ... LRG_98 LRG_99
## rowRanges metadata column names(0):
## colnames: NULL
## colData names(8): run experiment ... study sample
```

```
dim(se)
```

```
## [1] 63193    27
```

```
assay(se)[1:3,1:3]
```

```
##           [,1] [,2] [,3]
## ENSG000000000003 792 1064 444
## ENSG000000000005   4    1    2
## ENSG000000000419 294  282 164
```

```
dim(assay(se))
```

```
## [1] 63193    27
```

```
colData(se)[1:3,1:6]
```

```
## DataFrame with 3 rows and 6 columns
##           run experiment patient treatment      time submission
##  <character>  <factor> <factor>  <factor> <factor>    <factor>
## 1  SRR479052  SRX140503         1   Control    24h  SRA051611
## 2  SRR479053  SRX140504         1   Control    48h  SRA051611
## 3  SRR479054  SRX140505         1      DPN     24h  SRA051611
```

```
dim(colData(se))
```

```
## [1] 27  8
```

```
names(colData(se))
```

```
## [1] "run"          "experiment" "patient"     "treatment"  "time"
## [6] "submission"  "study"      "sample"
```

```
colData(se)$treatment
```

```
## [1] Control Control DPN      DPN      OHT      OHT      Control Control
## [9] DPN      DPN      DPN      OHT      OHT      OHT      Control Control
## [17] DPN      DPN      OHT      OHT      Control DPN      DPN      DPN
## [25] OHT      OHT      OHT
## Levels: Control DPN OHT
```

```
rowRanges(se)[1] # rowData has been replaced by rowRanges in latest GRanges package
```

```
## GRangesList object of length 1:
## $ENSG000000000003
## GRanges object with 17 ranges and 2 metadata columns:
##           seqnames           ranges strand | exon_id exon_name
##           <Rle>           <IRanges> <Rle> | <integer> <character>
## [1]           X [99883667, 99884983]     - |    664095 ENSE00001459322
## [2]           X [99885756, 99885863]     - |    664096 ENSE00000868868
## [3]           X [99887482, 99887565]     - |    664097 ENSE00000401072
## [4]           X [99887538, 99887565]     - |    664098 ENSE00001849132
## [5]           X [99888402, 99888536]     - |    664099 ENSE00003554016
## ...           ...           ...     ... |    ...
## [13]          X [99890555, 99890743]     - |    664106 ENSE00003512331
## [14]          X [99891188, 99891686]     - |    664108 ENSE00001886883
## [15]          X [99891605, 99891803]     - |    664109 ENSE00001855382
## [16]          X [99891790, 99892101]     - |    664110 ENSE00001863395
## [17]          X [99894942, 99894988]     - |    664111 ENSE00001828996
##
## -----
## seqinfo: 580 sequences (1 circular) from an unspecified genome
```

```
class(rowRanges(se))
```

```
## [1] "GRangesList"
## attr(,"package")
## [1] "GenomicRanges"
```

```
length(rowRanges(se))
```

```
## [1] 63193
```

```
head(rownames(se))
```

```
## [1] "ENSG000000000003" "ENSG000000000005" "ENSG000000000419" "ENSG000000000457"
## [5] "ENSG000000000460" "ENSG000000000938"
```

```
metadata(rowRanges(se))
```

```
## $genomeInfo
## $genomeInfo$`Db type`
## [1] "TranscriptDb"
##
## $genomeInfo$`Supporting package`
## [1] "GenomicFeatures"
##
## $genomeInfo$`Data source`
## [1] "BioMart"
##
## $genomeInfo$Organism
## [1] "Homo sapiens"
##
## $genomeInfo$`Resource URL`
## [1] "www.biomart.org:80"
##
## $genomeInfo$`BioMart database`
## [1] "ensembl"
##
## $genomeInfo$`BioMart database version`
## [1] "ENSEMBL GENES 72 (SANGER UK)"
##
## $genomeInfo$`BioMart dataset`
## [1] "hsapiens_gene_ensembl"
##
## $genomeInfo$`BioMart dataset description`
## [1] "Homo sapiens genes (GRCh37.p11)"
##
## $genomeInfo$`BioMart dataset version`
## [1] "GRCh37.p11"
##
## $genomeInfo$`Full dataset`
## [1] "yes"
##
## $genomeInfo$`miRBase build ID`
## [1] NA
##
## $genomeInfo$transcript_nrow
## [1] "213140"
##
## $genomeInfo$exon_nrow
## [1] "737783"
##
## $genomeInfo$cds_nrow
## [1] "531154"
##
## $genomeInfo$`Db created by`
## [1] "GenomicFeatures package from Bioconductor"
##
## $genomeInfo$`Creation time`
## [1] "2013-07-30 17:30:25 +0200 (Tue, 30 Jul 2013)"
##
## $genomeInfo$`GenomicFeatures version at creation time`
```

```
## [1] "1.13.21"
##
## $genomeInfo$`RSQLite version at creation time`
## [1] "0.11.4"
##
## $genomeInfo$DBSCHEMAVERSION
## [1] "1.0"
```

```
exptData(se)$MIAME
```

```
## Experiment data
##   Experimenter name: Felix Haglund
##   Laboratory: Science for Life Laboratory Stockholm
##   Contact information: Mikael Huss
##   Title: DPN and Tamoxifen treatments of parathyroid adenoma cells
##   URL: http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE37211
##   PMIDs: 23024189
##
##   Abstract: A 251 word abstract is available. Use 'abstract' method.
```

```
abstract(exptData(se)$MIAME)
```

```
## [1] "Primary hyperparathyroidism (PHPT) is most frequently present in postmenopausal
women. Although the involvement of estrogen has been suggested, current literature indi
cates that parathyroid tumors are estrogen receptor (ER) alpha negative. Objective: The
aim of the study was to evaluate the expression of ERs and their putative function in p
arathyroid tumors. Design: A panel of 37 parathyroid tumors was analyzed for expression
and promoter methylation of the ESR1 and ESR2 genes as well as expression of the ERalph
a and ERbeta1/ERbeta2 proteins. Transcriptome changes in primary cultures of parathyroi
d adenoma cells after treatment with the selective ERbeta1 agonist diarylpropionitrile
(DPN) and 4-hydroxytamoxifen were identified using next-generation RNA sequencing. Resu
lts: Immunohistochemistry revealed very low expression of ERalpha, whereas all informat
ive tumors expressed ERbeta1 (n = 35) and ERbeta2 (n = 34). Decreased nuclear staining
intensity and mosaic pattern of positive and negative nuclei of ERbeta1 were significan
tly associated with larger tumor size. Tumor ESR2 levels were significantly higher in f
emale vs. male cases. In cultured cells, significantly increased numbers of genes with
modified expression were detected after 48 h, compared to 24-h treatments with DPN or
4-hydroxytamoxifen, including the parathyroid-related genes CASR, VDR, JUN, CALR, and O
RAI2. Bioinformatic analysis of transcriptome changes after DPN treatment revealed sign
ificant enrichment in gene sets coupled to ER activation, and a highly significant simi
larity to tumor cells undergoing apoptosis. Conclusions: Parathyroid tumors express ERb
eta1 and ERbeta2. Transcriptional changes after ERbeta1 activation and correlation to c
linical features point to a role of estrogen signaling in parathyroid function and dise
ase."
```

Footnotes

For more information about the `GenomicRanges` package, check out the PLOS Comp Bio paper, which the authors of `GenomicRanges` published: [Get it from here](http://www.ploscompbiol.org/article/info%3Adoi%2F10.1371%2Fjournal.pcbi.1003118)

(<http://www.ploscompbiol.org/article/info%3Adoi%2F10.1371%2Fjournal.pcbi.1003118>)

For more information on *SummarizedExperiment*: go here
(<http://www.nature.com/nmeth/journal/v12/n2/abs/nmeth.3252.html>)

Also the software vignettes have a lot of details about the functionality. Check out “An Introduction to Genomic Ranges Classes”. All of the vignette PDFs are available here
(<http://www.bioconductor.org/packages/release/bioc/html/GenomicRanges.html>).

EXERCISES

If you have not done so already, install the following library which is available from github repository:

```
library(devtools)
install_github("genomicsclass/GSE5859Subset")
```

Load the library, load the data it contains, and note that three tables are loaded:

```
library(GSE5859Subset)
data(GSE5859Subset)
dim(geneExpression)
```

```
## [1] 8793 24
```

```
dim(sampleInfo)
```

```
## [1] 24 4
```

```
dim(geneAnnotation)
```

```
## [1] 8793 4
```

This data represents a subset of a larger microarray experiment see here
(<http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE5859>) with GEO id GSE5859. We used it extensively in course 3. Note that these three tables are related, although nothing in our R sessions tells us they are. We are going to create an object that keeps them together and organized.

Question 3.3.1

What Bioconductor class should we use to do this? *ExpressionSet*

Question 3.3.2

The `geneExpression` contains the microarray measurements, `sampleInfo` provides sample information and `geneAnnotation` provides feature information. Read the help file for the `ExpressionSet` class and determine which of the following best describes the best way to use it in this context:

We can see that the elements `geneExpression` is the `assayData`, `sampleInfo` is the `phenoData`, and `geneAnnotation` is the `featureData`.

Question 3.3.3

Before creating this object we must make sure that the rows of `sampleInfo` match the columns of `geneExpression` and that the rows of `geneAnnotation` match the rows of `geneExpression`. This is an important step in which the data creators and the analysts must work closely together to get right. In this particular case, the tables have identifiers that can be used:

```
identical(colnames(geneExpression),sampleInfo$filename)
```

```
## [1] TRUE
```

```
identical(rownames(geneExpression),geneAnnotation$PROBEID)
```

```
## [1] TRUE
```

However, we only know these are the correct columns/rows to check because the data creator (in this case the instructor) told us so. In the `ExpressionSet` object we assure this connection is established by forcing the `rownames` of the `assayData` to match the `rownames` of `featureData` and the the `colnames` of `assayData` to match the `rownames` of `phenoData`.

Now lets create the objects we will use in `phenoData` slot by applying the `AnnotatedDataFrame` function to the appropriate table. Call this object `pd`, make sure to give it `rownames` that match the `colnames` of `geneExpression`

```
pd = AnnotatedDataFrame(sampleInfo)
rownames(pd) = colnames(geneExpression)
pd = pd[, colnames(pd)!="filename"] ##redundant
```

Once you are done creating these objects, report the value you obtain when you type:

```
pData(pd)[ "GSM136530.CEL.gz", "date" ]
```

```
## [1] "2005-06-27"
```

The reason we use the class `AnnotatedDataFrame`, as opposed to just using the data frames, is to encourage users to describe the variables represented in the table. [learn more here:](#)

```
?AnnotatedDataFrame
```

```
## starting httpd help server ... done
```

Question 3.3.4

```
varLabels(pd)
```

```
## [1] "ethnicity" "date"      "group"
```

```
dimLabels(pd)
```

```
## [1] "rowNames"  "columnNames"
```

Question 3.3.5

Note the `ExpressionSet` has a slot for a table describing the genes. This is not commonly used when the data comes from an array for which Bioconductor has an annotation package as it is more efficient to carry just one copy of this information as opposed to adding it to each object. Here we will create an `AnnotatedDataFrame` version of `geneAnnotation`.

To latter, add to `featureData` slot of an `ExpressionSet`. Use the `AnnotatedDataFrame` function to create an object `fd`. Make sure the `rownames` match the `rownames` of `geneExpression`

```
fd = AnnotatedDataFrame(geneAnnotation)
rownames(fd) = geneAnnotation$PROBEID
fd = fd[,colnames(fd) != "PROBEID"] ## redundant
```

Run the command

```
pData(fd)["204810_s_at", "CHR"]
```

```
## [1] "chr19"
```

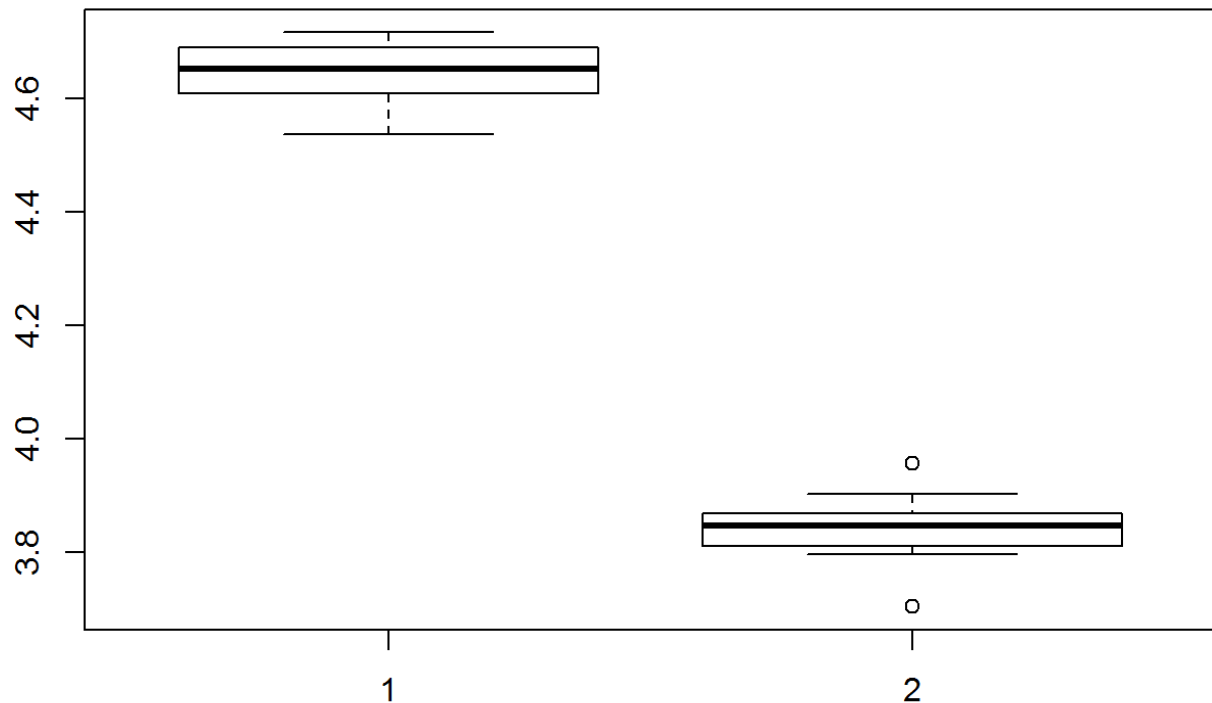
Question 3.3.6

Now we are ready to create our `ExpressionSet` object. Use the `ExpressionSet` function and populate it with `geneExpression`, `pd`, and `fd`. The latter two were created in question 3.3.2. Call the new object `eset`.

```
eset = ExpressionSet(geneExpression, phenoData = pd, featureData = fd)
```

After creating `eset` run the following lines. What is the different in medians produced by the last line?

```
ind1 = which( featureData(eset)$CHR=="chrY" )
ind2 = pData(eset)$group==1
femaleY = colMeans(exprs(eset)[ind1, ind2])
maleY = colMeans(exprs(eset)[ind1, !ind2])
boxplot(maleY, femaleY)
```



```
median(maleY)-median(femaleY)
```

```
## [1] 0.8049265
```

Now that you have the data organized in such a way that you can access the different components from the same object:

```
dim(pData(eset))
```

```
## [1] 24 3
```

```
dim( exprs(eset) )
```

```
## [1] 8793 24
```

```
dim( featureData( eset ) )
```

```
## featureNames featureColumns
##          8793           3
```

We also know (because we created the original datasets) that these measurements were made with the `hgfocus` array, thus we can add this as well:

```
annotation(eset) = "hgfocus"
```

Question 3.3.7

Suppose that now you want to take a subset of this data that considers just the first 10 features and first 5 samples. Which of the following is the best way to create such an object?

```
eset[1:10,1:5]
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 10 features, 5 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: GSM136508.CEL.gz GSM136530.CEL.gz ...
##     GSM136566.CEL.gz (5 total)
##   varLabels: ethnicity date group
##   varMetadata: labelDescription
## featureData
##   featureNames: 1007_s_at 1053_at ... 1438_at (10 total)
##   fvarLabels: CHR CHRLOC SYMBOL
##   fvarMetadata: labelDescription
## experimentData: use 'experimentData(object)'
## Annotation: hgfocus
```

Question 3.3.8

Now we are going to work with the `SummarizedExperiment` class. In the essence, this class is very similar to `ExpressionSet`. The main difference is that `SummarizedExperiment` is meant to store measurements associated with genome locations. The location information is stored as a `GRanges` object. Unfortunately for the user, `SummarizedExperiment` uses a different set of arguments to store essentially the same tables. Below is an example of how we would construct a `SummarizedExperiment` for the `eset` we created in the Question 3.3.4

Here is an example of how one can use Bioconductor databases to match IDs from the manufacturer to the IDs from the public repositories, and then use this to create a `GRanges` object.

```
library(Homo.sapiens)
```

```
## Loading required package: AnnotationDbi
## Loading required package: OrganismDbi
## Loading required package: GenomicFeatures
## Loading required package: GO.db
## Loading required package: DBI
##
## Loading required package: org.Hs.eg.db
##
## Loading required package: TxDb.Hsapiens.UCSC.hg19.knownGene
```

```
genes = genes(Homo.sapiens)
```

```
library(hgfocus.db)
```

```
##
```

```
## get the entrezIDs associated with the probe ids for this array.
```

```
map = select(hgfocus.db, keys = featureNames(eset), columns = "ENTREZID", keytype = "PROBEID")
```

```
## since we obtain a multiple map, pick the first (match does this automatically)
```

```
index1 = match(featureNames(eset),map[,1]) ##pick the first
```

```
## now use this to map to the genes GRanges
```

```
index2 = match(map[index1,2], as.character(mcols(genes)$GENEID))
```

```
## we will remove the NAs
```

```
index3 = which(!is.na(index2))
```

```
index2 = index2[index3] ##remove if we don't have a match
```

```
## Subset the objects to map
```

```
genes = genes[index2,]
```

```
neweset = eset[index3,]
```

Now that we are ready to create `SummarizedExperiment` from our `eset`.

```
se = SummarizedExperiment(assays=exprs(neweset),
                          rowRanges=genes,
                          colData=DataFrame(pData(neweset)))
```

Now we can access the expression data with `assay` and the gene locations with `GRanges`.

```
dim(assay(se))
```

```
## [1] 8494 24
```

```
length(granges(se))
```

```
## [1] 8494
```

How many genes have their TSS (hint: use `resize` function) on the first 50 million bases of chromosome 1.

```
tss = start(resize( granges(se),1))
sum( tss < 50*10^6 & seqnames( se)=="chr1" )
```

[1] 265

Note that can easily plot, for example, differential expression across the genome

```
### we will re-order se
se = se[order(granges(se)),]
ind = se$group==1
de = rowMeans( assay(se)[,ind])-rowMeans( assay(se)[,!ind])
chrs = unique( seqnames(se))
library(rafalib)
mypar(3,2)
for(i in c(1:4)){
  ind = which(seqnames( se) == chrs[i])
  plot(start(se)[ind], de[ind], ylim=c(-1,1),main=as.character(chrs[i]))
  abline(h=0)
}
##now X and Y
for(i in 23:24){
  ind = which(seqnames( se) == chrs[i])
  ##note we use different ylims
  plot(start(se)[ind], de[ind], ylim=c(-5,5),main=as.character(chrs[i]))
  abline(h=0)
}
```

